# CulinaryGraph

**Mustafa Erötgen**
2024719141

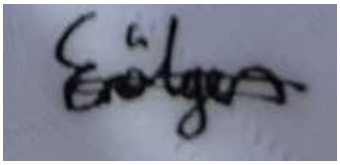
**SWE 573 — Software Development Practice**
Date: 16 May 2026

**Project name:** CulinaryGraph
**Deployment URL:** https://culinary-service-727322696060.europe-west10.run.app/
**Git repository URL:** https://github.com/Erotgen/2024719141-SWE-573-Software-Development-Practice
**Git tag version URL (v0.9):** https://github.com/Erotgen/2024719141-SWE-573-Software-Development-Practice/releases/tag/v0.9

## HONOR CODE

Related to the submission of all the project deliverables for the Swe573 Spring 2026 semester project reported in this report, I Mustafa Erötgen declare that: - I am a student in the Software Engineering MS program at Bogazici University and am registered for Swe573 course during the Spring 2026 semester. - All the material that I am submitting related to my project (including but not limited to the project repository, the final project report, and supplementary documents) have been exclusively prepared by myself. - I have prepared this material individually without the assistance of anyone else with the exception of permitted peer assistance, which I have explicitly disclosed in this report.

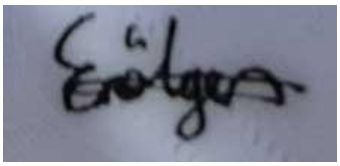
Mustafa Erötgen



## Declaration of Third-Party Software

All source code in this project was written by the author. The following third-party open-source dependencies are used in accordance with their licenses; no third-party code was copied into the repository.

Mustafa Erötgen



## Test Credentials for the Deployed System

The deployed application at the URL above is seeded with the accounts below.

**Administrator (staff + superuser)**
Email: admin@example.com
Password: Aa123456

**Regular user 1 (Turkiye)**
Email: nerminisik@example.com
Password: Aa123456

**Regular user 2 (Lebanon)**
Email: sarah@example.com
Password: Aa123456

# Table of Contents

## Contents

## 1. AI Use Declaration

AI-assisted tools were used during the project. Each use is disclosed below with the tool, task, level of reliance, and how the output was validated. All produced artefacts were reviewed, tested, and integrated by the author; no AI output was committed without inspection.

**Claude Sonnet**
Task: Drafting MANUAL_TEST_PLAN.md (~80 UAT cases across 14 feature areas).
Reliance: Major generation
Validation: I walked every case end-to-end in the deployed instance, adjusted expected results that did not match observed behaviour. Then I tested each case manually.

**Claude Sonnet**
Task: AI tools were used to identify the cause of various unresolved errors.
Reliance: Partial generation
Validation: The reason for each change was identified and then tested.

**Claude Sonnet**
Task: Inline completion of repetitive Django template boilerplate.
Reliance: Partial generation
Validation: No template was committed without review and a manual test.

**Claude Sonnet**
Task: Generating deployment Powershell Scripts.
Reliance: Partial generation

Validation: After running the deployment script, I checked to see if the target had been achieved.

**Claude Sonnet**
Task: Generating every recipe, ingredient, technique data in given format.
Reliance: Major generation.
Validation: Context generated but manually added.

**Claude Sonnet**
Task: Drafting the Region master list (country names) used in schema.sql seeds.
Reliance: Major generation
Validation: List manually reduced to ISO-recognised country names; duplicates removed by me.

**Claude Sonnet**
Task: Drafting copy text for empty-state messages, error banners, and confirmation dialogs, welcoming messages across the templates.
Reliance: Major generation
Validation: I edited the tone to match the project voice.

**Claude Sonnet**
Task: A Docker file was generated after the dockerize process failed due to errors.
Reliance: Major generation
Validation: The purpose of each line of code has been identified and corrected.

# 2. Overview

## 2.1 What is it?

CulinaryGraph is a region-aware recipe sharing platform that ties every recipe, ingredient, and technique to a heritage, supports community moderation through edit proposals and reports, and offers cross-region exploration via an interactive map and side-by-side comparison view.

## 2.2 Target audience

- Home cooks who want to learn the regional context behind a dish.
- Culinary students researching how techniques diverge across cuisines.
- Food researchers and recipe developers gathering structured cross-region data.

# 3. Software Requirements Specification

## 3.1 Objective and scope

The purpose of this specification is to define the functional and non-functional requirements of CulinaryGraph and to keep all stakeholders aligned on what the system does and does not do.

CulinaryGraph is a web application where authenticated users share food-related content from their region (recipes, ingredients, techniques) and interact with content from other regions through likes, comments, edit proposals, reports, search, comparison, and a map view.

### 3.2 Functional requirements

- FR-1: Users register with first name, last name, email, and password. Email must be unique.
- FR-2: Users log in by email and password.
- FR-3: User profile is created on registration and stores: date of birth, gender, about me, country, and optional profile photo information.
- FR-4: User profile is complete only when the country, about me, date of birth, and gender are all set.
- FR-5: The user cannot add any entry until the profile is complete.
- FR-6: Country cannot be changed once set.
- FR-7: Users can update non-country profile fields and profile photo at any time.
- FR-8: Recipe creation requires: title, instructions, cultural significance, ingredients, techniques, location on the map, and a complete profile.
- FR-9: The author's profile region is auto-attached as the recipe country.
- FR-10: The recipe detail page shows: image, title, author, cook time, difficulty, country tags, like button, instructions, cultural significance, ingredients, techniques, and origin-location map.
- FR-11: Recipe page shows up to 60 recipes after filtering.
- FR-12: Ingredient creation requires: name and description.
- FR-13: Ingredient detail page shows: name, description.
- FR-14: Techniques creation requires: name, description, cultural significance, and optionally, necessary tools.
- FR-15: The author's profile region is auto-attached as the techniques' country.
- FR-16: The technique detail page shows: name, author, description, cultural significance, and necessary tools.
- FR-17: Users can like a recipe.
- FR-18: The recipe detail page displays a heart button as the total like count.
- FR-19: Users can comment on any entry (recipe, ingredient, technique).
- FR-20: Comments are likeable or dislikeable by any user.
- FR-21: Comments are ordered by creation time ascending, replies grouped under their parent.
- FR-22: Users can search any entry via the search button.
- FR-23: Recipes — filterable in the Recipe page by: title, author, country, difficulty, cook time min, ingredient name, technique name, instructions keyword, cultural significance keyword.
- FR-24: Ingredients — filterable in the Ingredient page by: name, author, description.
- FR-25: Techniques — filterable in the Technique page by: name, author, country, description keyword, cultural significance keyword, and necessary tools keyword.
- FR-26: A "Clear filters" link appears whenever any filter is active.

- FR-27: The author of an entry can edit it directly.
- FR-28: Non-authors submit changes on the entries; the system creates a pending proposal.
- FR-29: The entry author sees inbound proposals on the Proposals page and can approve or reject each with an optional decision note.
- FR-30: Approved proposals apply the entries; rejected proposals does nothing.
- FR-31: Proposers see the status of their own proposals on the same Proposals page.
- FR-32: Non-authors can report any entry via the Report button with a reason. A new Report is created with a pending status.
- FR-33: Staff can reply to the Report.
- FR-34: Staff can ban a user or delete an entry.
- FR-35: Culinary Road page has an interactive world map with one marker per recipe.
- FR-36: Marker popup shows: recipe image, title, cultural heritage, and a link to details.
- FR-37: Culinary Compare page lets users compare ≥2 entries of the same kind.
- FR-38: In the Culinary Compare page, tabs switch between Recipes, Ingredients, and Techniques. 39: FR-39: A multi-select picker lists all entries of the chosen kind, ordered by title; user picks 2+ and submits.
- FR-40: With <2 selected, the page shows a "pick at least two" error.
- FR-41: After the selection of the comparable objects, a comparison table appears.

### 3.3 Non-functional requirements

- **NFR-1:** Search results shall be returned within 2 seconds under nominal load.
- **NFR-2:** The system shall support up to 10,000 concurrent users.
- **NFR-3:** Database writes shall be replicated to at least two geographically distinct data centers.
- **NFR-4:** All data transmission between client and server shall use TLS 1.2 or higher.
- **NFR-5:** The system shall be fully usable on viewport widths from 320 px (mobile) to 2560 px (large desktop).

# 4. Design

## 4.1 Architectural style

CulinaryGraph follows Django's Model–Template–View (MVT) architecture in a single-process monolith. There is exactly one app (core/) holding the data model; views and URL routing live at the project root (view.py, url.py) for simplicity. The application is server-rendered with progressive enhancement (Leaflet maps and small inline JS for filter chips); there is no SPA frontend.

## 4.3 Data model

Eight Django models, all anchored on Django's built-in User. Polymorphic relationships (Comment, Report, EditProposal) are implemented with three nullable foreign keys (recipe / ingredient / technique) rather than GenericForeignKey, to retain DB-level referential integrity and simpler joins.

**Region**
Purpose: Country / region master list (seeded from schema.sql).
Key fields / relations: name (unique)

**Profile**
Purpose: Per-user metadata. Country is locked after first save (FR-4).
Key fields / relations: OneToOne(User); FK(Region); date_of_birth, gender, description, photo

**Recipe**
Purpose: Region-tagged recipe with geo-coordinates for the map.
Key fields / relations: FK(User author); M2M(Region, Ingredient, Technique, liked_by); latitude/longitude; cook_time; difficulty

**Ingredient**
Purpose: Cultural ingredient with description.
Key fields / relations: FK(User author); name; description; cultural_significance

**Technique**
Purpose: Cooking technique with tooling notes.
Key fields / relations: FK(User author); M2M(Region); necessary_tools

**Comment**
Purpose: Polymorphic comment with one-level threading and like/dislike.
Key fields / relations: FK(User); nullable FK(Recipe / Ingredient / Technique); self-FK parent; M2M liked_by, disliked_by

**EditProposal**
Purpose: Pending change submitted by a non-owner; JSONField payload + workflow.

Key fields / relations: FK(User proposer); nullable FK(Recipe / Ingredient / Technique); status (pending/approved/rejected); JSON payload

**Report**
Purpose: Abuse report with admin resolution workflow.
Key fields / relations: FK(User reporter); nullable FK targets; status (pending/resolved/dismissed); FK(User resolved_by)

## 5. Status of Requirements

### 5.1 Requirements

- **FR-1 — Completed**
  Requirement: Users register with first name, last name, email, and password. Email must be unique.
- **FR-2 — Completed**
  Requirement: Users log in by email and password.
- **FR-3 — Completed**
  Requirement: User profile is created on registration and stores date of birth, gender, About Me, country, and optional profile photo information.
- **FR-4 — Completed**
  Requirement: User profile is complete only when country, About Me, date of birth, and gender are all set.
- **FR-5 — Completed**
  Requirement: The user cannot add any entry until the profile is complete.
- **FR-6 — Completed**
  Requirement: Country cannot be changed once set.
- **FR-7 — Completed**
  Requirement: Users can update non-country profile fields and profile photo at any time.
- **FR-8 — Completed**
  Requirement: Recipe creation requires title, instructions, cultural significance, ingredients, techniques, location on the map, and a complete profile.
- **FR-9 — Completed**
  Requirement: The author's profile region is auto-attached as the recipe country.
- **FR-10 — Completed**
  Requirement: The recipe detail page shows image, title, author, cook time, difficulty, country tags, like button, instructions, cultural significance, ingredients, techniques, and origin-location map.
- **FR-11 — Completed**
  Requirement: Recipe page shows up to 60 recipes after filtering.
- **FR-12 — Completed**
  Requirement: Ingredient creation requires name and description.

- **FR-13 — Completed**
  Requirement: Ingredient detail page shows name and description.
- **FR-14 — Completed**
  Requirement: Techniques creation requires name, description, cultural significance, and optionally necessary tools.
- **FR-15 — Completed**
  Requirement: The author's profile region is auto-attached as the technique's country.
- **FR-16 — Completed**
  Requirement: The technique detail page shows name, author, description, cultural significance, and necessary tools.
- **FR-17 — Completed**
  Requirement: Users can like a recipe.
- **FR-18 — Completed**
  Requirement: The recipe detail page displays a heart button with the total like count.
- **FR-19 — Completed**
  Requirement: Users can comment on any entry (recipe, ingredient, technique).
- **FR-20 — Completed**
  Requirement: Comments are likeable or dislikeable by any user.
- **FR-21 — Completed**
  Requirement: Comments are ordered by creation time ascending, replies grouped under their parent.
- **FR-22 — Completed**
  Requirement: Users can search any entry via the search button.
- **FR-23 — Completed**
  Requirement: Recipes are filterable in the Recipe page by title, author, country, difficulty, cook time min, ingredient name, technique name, instructions keyword, and cultural significance keyword.
- **FR-24 — Completed**
  Requirement: Ingredients are filterable in the Ingredient page by name, author, and description.
- **FR-25 — Completed**
  Requirement: Techniques are filterable in the Technique page by name, author, country, description keyword, cultural significance keyword, and necessary tools keyword.
- **FR-26 — Completed**
  Requirement: A 'Clear filters' link appears whenever any filter is active.
- **FR-27 — Completed**
  Requirement: The author of an entry can edit it directly.
- **FR-28 — Completed**
  Requirement: Non-authors submitting changes on an entry result in the system creating a pending proposal.

- **FR-29 — Completed**
  Requirement: The entry author sees inbound proposals on the Proposals page and can approve or reject each with an optional decision note.
- **FR-30 — Completed**
  Requirement: Approved proposals apply the entries; rejected proposals does nothing.
- **FR-31 — Completed**
  Requirement: Proposers see the status of their own proposals on the same Proposals page.
- **FR-32 — Completed**
  Requirement: Non-authors can report any entry via the Report button with a reason. A new Report is created with a pending status.
- **FR-33 — Completed**
  Requirement: Staff can reply to the Report.
- **FR-34 — Completed**
  Requirement: Staff can ban a user or delete an entry.
- **FR-35 — Completed**
  Requirement: Culinary Road page has an interactive world map with one marker per recipe.
- **FR-36 — Completed**
  Requirement: Marker popup shows recipe image, title, cultural heritage, and a link to details.
- **FR-37 — Completed**
  Requirement: Culinary Compare page lets users compare two or more entries of the same kind.
- **FR-38 — Completed**
  Requirement: In the Culinary Compare page, tabs switch between Recipes, Ingredients, and Techniques.
- **FR-39 — Completed**
  Requirement: A multi-select picker lists all entries of the chosen kind, ordered by title; the user picks two or more and submits.
- **FR-40 — Completed**
  Requirement: With fewer than two selected, the page shows a 'pick at least two' error.
- **FR-41 — Completed**
  Requirement: After the selection of the comparable objects, a comparison table appears.

## 6. Status of Deployment

- **Deployed:** Yes
- **Public URL:** https://culinary-service-727322696060.europe-west10.run.app/
- **Region:** europe-west10 (Berlin)
- **Platform:** Google Cloud Run
- **Dockerized?:** Yes
- **Database:** Google Cloud SQL for MySQL 8

## 7. Installation Instructions

### 7.1 System requirements

- **OS:** Linux, macOS, or Windows 10 / 11
- **Python:** 3.12 or newer (used in Dockerfile); 3.11+ works locally
- **MySQL:** 8.0 or newer
- **Docker:** Docker Desktop 4.x or Docker Engine 24+ (only if using the Docker path)
- **Memory:** 1 GiB free during build (mysqlclient compiles a C extension)
- **Disk:** ~600 MiB for the Docker image; ~150 MiB for a bare-metal venv

### 7.2 Required environment variables

- **SECRET_KEY:** required: Yes (prod); example: a-long-random-string
- **DEBUG:** required: No; example: 1 (dev) / 0 (prod)
- **ALLOWED_HOSTS:** required: No (dev); example: * or mysite.com,api.mysite.com
- **DB_NAME:** required: Yes; example: CulinaryGraph
- **DB_USER:** required: Yes; example: root
- **DB_PASSWORD:** required: Yes; example: your-password
- **DB_HOST:** required: Yes; example: 127.0.0.1, db, or /cloudsql/PROJECT:REGION:INSTANCE
- **DB_PORT:** required: No; example: 3306
- **GS_BUCKET_NAME:** required: No; example: culinarygraph-media (activates GCS storage)

### 7.3 Option A: Docker Compose (recommended)

1. Install Docker Desktop and start it.
2. From the project root, create a .env file from .env.example and edit it (set DB_HOST=db).
3. Build and start the stack:

```
docker compose up --build
```

4. In a second terminal, apply migrations and load the seed:

```
docker compose exec web python manage.py migrate
docker compose exec -T db mysql -uroot -p"$DB_PASSWORD" CulinaryGraph < schema.sql
```

5. Open http://127.0.0.1:8000/ in your browser.

### 7.4 Option B: Bare-metal Python + MySQL

6. Install MySQL 8 locally and create the database CulinaryGraph.
7. From the project root, create a virtual environment and install dependencies:

```
python -m venv .venv
.venv\Scripts\activate          (Windows)
source .venv/bin/activate         (Linux / macOS)
pip install -r requirements.txt
```

8. Create .env (see 7.2) and load the schema:

```
mysql -u root -p CulinaryGraph < schema.sql
```

9. Start the development server:

```
python manage.py runserver 127.0.0.1:8000
```

### 7.5 Option C: Deploy to Google Cloud Run (production)

10. Create a Cloud SQL MySQL 8 instance with an empty CulinaryGraph database, and bootstrap the schema:

```
gcloud sql connect YOUR_INSTANCE --user=root --database=CulinaryGraph < schema.sql
```

11. Deploy from source in one command:

```
gcloud run deploy culinary-service \
  --source . \
  --region europe-west10 \
  --allow-unauthenticated \
  --add-cloudsql-instances PROJECT:REGION:INSTANCE \
  --set-env-vars
DB_NAME=CulinaryGraph,DB_USER=root,DB_PASSWORD=...,DB_HOST=/cloudsql/PROJECT:REGION:
INSTANCE,SECRET_KEY=...,DEBUG=0,ALLOWED_HOSTS=*
```

After deployment, create the administrator user (one-time):

```
gcloud sql connect YOUR_INSTANCE --user=root --database=CulinaryGraph <
scripts/insert_admin_user.sql
```

# 8. API Documentation

## 8.1 Live API documentation

- Swagger UI: https://culinary-service-727322696060.europe-west10.run.app/api/docs/

## 8.2 Endpoint inventory

- **Auth:** POST /register/, POST /login/, GET /logout/
- **Profile:** GET/POST /profile/
- **Recipes:** GET /recipes/, GET/POST /recipes/add/, GET /recipes/{pk}/, POST /recipes/{pk}/like/, GET/POST /recipes/{pk}/edit/
- **Techniques:** GET /techniques/, GET/POST /techniques/add/, GET /techniques/{pk}/, GET/POST /techniques/{pk}/edit/
- **Ingredients:** GET /ingredients/, GET/POST /ingredients/add/, GET /ingredients/{pk}/, GET/POST /ingredients/{pk}/edit/
- **Comments:** POST /comments/{kind}/{pk}/add/, POST /comments/{pk}/like/, POST /comments/{pk}/dislike/
- **Edit Proposals:** GET /proposals/, POST /proposals/{pk}/decide/
- **Reports:** GET/POST /reports/{kind}/{pk}/submit/, GET /reports/, POST /reports/{pk}/action/
- **Discovery:** GET /culinary-road/, GET /culinary-compare/, GET /search/
- **Docs:** GET /openapi.yaml, GET /api/docs/

## 8.3 Three usage scenarios

All three scenarios below exercise functionality added after Milestone 2: the multi-filter recipe search with fuzzy-match autocorrect, the Culinary Compare side-by-side view, and the non-owner edit-proposal workflow.

### Scenario 1 — Fuzzy multi-filter recipe search

Endpoint: GET /recipes/

A researcher looking for slow-cooked Spanish recipes mistypes the ingredient 'saffron' as 'saffrn'; the server substring-matches first, falls back to difflib.get_close_matches, and returns the autocorrected hit alongside an ingredient_corrected hint that the UI surfaces.

```
curl -s -G 'https://culinary-service-727322696060.europe-west10.run.app/recipes/' \
  --data-urlencode 'region=Spain' \
  --data-urlencode 'difficulty=Hard' \
  --data-urlencode 'cook_min=30' \
  --data-urlencode 'ingredient=saffrn'
```

Representative response (server-rendered HTML page; abbreviated as JSON for readability — the test client returns the same data via response.context):

```
{
  "ingredient_corrected": "Saffron",
  "any_filter": true,
```

```
  "filters": {
    "region": "Spain", "difficulty": "Hard",
    "cook_min": "30", "ingredient": "saffrn"
  },
  "recipes": [
    {
      "id": 17,
      "title": "Paella",
      "author": "chef@example.com",
      "regions": ["Spain"],
      "ingredients": ["Saffron"],
      "difficulty": "Hard",
      "cook_time": 60
    }
  ]
}
```


### Scenario 2 — Side-by-side comparison of three ingredients

Endpoint: GET /culinary-compare/

A culinary student selects three ingredients to compare their authors and cultural significance side-by-side. The view dispatches on `kind` and conditionally prefetches `regions` only for kinds that have it (the original implementation incorrectly prefetched it for Ingredients too, causing a 500 — fixed in view.py).

```
curl -s -G 'https://culinary-service-727322696060.europe-west10.run.app/culinary-
compare/' \
  -b cookies.txt \
  --data-urlencode 'kind=ingredient' \
  --data-urlencode 'ids=3' \
  --data-urlencode 'ids=7' \
  --data-urlencode 'ids=12'
```

Representative response (test-client context, abbreviated):


```
{
  "kind": "ingredient",
  "selected_ids": [3, 7, 12],
  "selected": [
    {"id": 3,  "name": "Saffron"},
    {"id": 7,  "name": "Sumac"},
    {"id": 12, "name": "Harissa"}
  ],
  "rows": [
    {"label": "Name",    "values": ["Saffron", "Sumac", "Harissa"]},
    {"label": "Author",  "values": ["chef@example.com", "amir@example.com",
"yasmine@example.com"]},
    {"label": "Description",            "values": ["Crocus stigmas", "Tart berry
powder", "Chilli paste"]},
    {"label": "Cultural Significance", "values": ["...", "...", "..."]}
  ]
}
```

### Scenario 3 — Non-owner edit proposal followed by owner approval

Endpoints: POST /recipes/{pk}/edit/ and POST /proposals/{pk}/decide/

User B (the non-owner) suggests an improved title for User A's recipe. The server creates a pending EditProposal carrying only the diffed fields. User A later opens /proposals/ and approves it; the recipe is updated in place and the proposal is marked approved with a decision note.

```
# 1. user_b proposes a title change
curl -s -X POST 'https://culinary-service-727322696060.europe-
west10.run.app/recipes/17/edit/' \
  -b user_b_cookies.txt \
  --data-urlencode 'title=Paella Valenciana' \
  --data-urlencode 'content=Cook rice with saffron …' \
  --data-urlencode 'cultural_significance=Valencian heritage dish' \
  --data-urlencode 'cook_time=60' \
  --data-urlencode 'difficulty=Hard' \
  --data-urlencode 'note=More accurate regional name'

# 2. user_a (the owner) approves
curl -s -X POST 'https://culinary-service-727322696060.europe-
west10.run.app/proposals/42/decide/' \
  -b user_a_cookies.txt \
  --data-urlencode 'decision=approve' \
  --data-urlencode 'decision_note=Thanks, agreed.'
```

Representative DB state after approval:

```
{
  "proposal": {
    "id": 42,
    "proposer": "user_b@test.com",
    "recipe_id": 17,
    "payload": {"title": "Paella Valenciana"},
    "status": "approved",
    "decision_note": "Thanks, agreed.",
    "decided_at": "2026-05-15T22:14:03Z"
  },
  "recipe": {
    "id": 17,
    "title": "Paella Valenciana",
    "author": "user_a@test.com"
  }
}
```

## 9. User Manual

### 9.1 First-time setup

12. Visit the deployment URL and click Join.
13. Provide first name, last name, email, and a password. Sign in.

14. You are taken to the Profile page. Pick your country (this is locked once saved — FR-4), write a short About Me, set your date of birth and gender, optionally upload a profile photo, and save.
15. You are now ready to publish recipes, techniques, and ingredients.

### 9.2 Contributing content

- Recipes: Dashboard → + Add Recipe. Title, instructions, cultural significance, a map pin and a cook time are required. Optional: difficulty, image upload, ingredient and technique tags.
- Techniques: /techniques/add/ — name, description, cultural significance required; necessary_tools optional.
- Ingredients: /ingredients/add/ — name and description required.

### 9.3 Discovery

- Browse: /recipes/, /techniques/, /ingredients/ all support per-field filters with substring matching and fuzzy-typo autocorrection.
- Map: /culinary-road/ shows every geo-tagged recipe as a Leaflet marker; click for a preview popup.
- Compare: /culinary-compare/ lets you pick two or more entries of the same kind (recipe, ingredient, or technique) and renders a side-by-side table.
- Global search: /search/ runs the same query against titles, descriptions, and cultural significance of all three content types.

### 9.4 Community interactions

- Like a recipe: heart button on the detail page (toggle).
- Comment / reply: textarea at the bottom of any detail page. Replies are one level deep.
- Like / dislike a comment: per-comment buttons (mutually exclusive).
- Suggest an edit on someone else's entry: Edit button on the detail page → form submits as a pending EditProposal.
- Report problematic content: Report button on any detail page; reasons are surfaced to moderators in /reports/.

### 9.5 Administrator manual

- Sign in with the admin account (see Test Credentials section).
- /reports/ — pending reports at the top with three actions: Delete Entry, Ban User, Dismiss with a note. Resolved / dismissed history below.
- /proposals/ — only visible for proposals that target your own entries. Approve applies the diff atomically; Reject preserves the record with your decision_note.

## 10. End-to-End Use Case: A Cross-Border Kunafah Exchange

Actors:

- Nermin (registered in Turkey) — content author.
- Sarah (registered in Lebanon) — community contributor.
- Admin — moderator.

**Story:**

16. Nermin registers, completes her profile with country=Türkiye, and adds her family's kunafah recipe with a map pin in Hatay, two ingredients and one technique. The recipe shows up under My Latest Entries and on the Culinary Road map.
17. Sarah signs up, completes his profile with country=Lebanon. She browses /recipes/?region=Türkiye, opens Nermin's recipe, and likes it.
18. Sarah notices the recipe says 'fry for 5 minutes', which conflicts with the way he learned it. He clicks Edit, changes the time to 4 minutes, adds a note 'usually 4 min in my family', and submits.
19. The server detects Sarah is not the owner and creates a pending EditProposal carrying only the changed field (cook_time and the note); Nermin's recipe on disk is unchanged. Sarah is redirected back with ?proposed=1.
20. Nermin opens /proposals/, sees the diff (5 → 4) with Sarah's note, agrees, and clicks Approve with the message 'Good catch'. The recipe is updated atomically and the proposal is marked approved with decided_at populated.
21. Sarah then opens /culinary-compare/?kind=recipe, multi-selects Sarah's recipe alongside two Lebanese kunafah recipes, and reviews differences across regions, ingredients, and cultural significance in one table.
22. A third user posts a recipe with offensive language. Sarah clicks Report on the recipe and submits a reason. The admin opens /reports/, sees the pending report, clicks Delete Entry with the note 'Hate speech'; the offending recipe is removed and an audit-trail Report row is created.

Outcome: a single piece of content has been authored, discovered cross-region, refined by community input, compared against peers in other regions, and moderated by an admin — exercising the registration, profile, recipe, edit-proposal, comparison, and report subsystems end-to-end.

## 11. Test Plan and Results

**AUTH-01**

Steps: Visit /, click Join, fill first_name, last_name, valid email, password >= 1 char, submit.
Expected (P): Green banner: 'Account created! You can log in now.'
**Result: PASS**

**AUTH-02 (N)**

Steps: Repeat AUTH-01 with the same email.
Expected (P): Red banner: 'An account with this email already exists.'
**Result: PASS**

**AUTH-03 (N)**

Steps: Submit register form leaving any field blank.
Expected (P): Red banner: 'All fields are required.'
**Result: PASS**

**AUTH-04**

Steps: Visit /login/, submit registered email + correct password.
Expected (P): Redirected to /entry/.
**Result: PASS**

**AUTH-05 (N)**

Steps: Visit /login/, submit wrong password.
Expected (P): Red banner: 'Invalid email or password.' (status 200, stays on /login/).
**Result: PASS**

**AUTH-06 (N)**

Steps: Visit /login/, submit unknown email.
Expected (P): Same error as AUTH-05.
**Result: PASS**

**AUTH-07**

Steps: While logged in, click Logout.
Expected (P): Redirected to /, header now shows Join / Sign in.
**Result: PASS**

**AUTH-08**

Steps: While logged in, visit /login/ directly.
Expected (P): Redirected to /entry/ (already authenticated).
**Result: PASS**

**AUTH-09 (N)**

Steps: While logged out, visit /entry/, /profile/, /recipes/add/, /proposals/, /reports/, /culinary-compare/, /culinary-road/, /search/.

Expected (P): Each redirects to /login/?next=…
**Result: PASS**

**PROF-01**
Steps: Visit /profile/ for the first time.
Expected (P): Empty form, all fields required, region/country dropdown populated.
**Result: PASS**

**PROF-02 (N)**
Steps: Save with any field empty.
Expected (P): Red banner: 'Required: country, about me, date of birth, gender.' (list reflects missing fields).
**Result: PASS**

**PROF-03**
Steps: Fill all fields, pick France, attach photo, save.
Expected (P): Green banner 'Profile saved successfully.'; photo visible after reload.
**Result: PASS**

**PROF-04**
Steps: Reload /profile/; in the country dropdown try to select Italy and save.
Expected (P): Country selector should be disabled/locked OR save silently keeps France (FR-6 enforcement at core/models.py).
**Result: PASS**

**PROF-05**
Steps: Update only the description / DOB / gender / photo.
Expected (P): Green banner; region still France.
**Result: PASS**

**PROF-06**
Steps: Visit /recipes/add/ before completing profile (use a freshly registered user).
Expected (P): Redirected to /profile/?required=1, page shows: 'Complete your profile to start adding recipes.'
**Result: PASS**

**ENT-01**
Steps: After login, land on /entry/.
Expected (P): 'Your region: France' header; 'My Latest Entries' panel (empty for new user); 'From the Community' panel populated from existing recipes.
**Result: PASS**

**ENT-02**
Steps: After adding a recipe (REC-01), revisit /entry/.

Expected (P): New recipe appears under My Latest Entries.
**Result: PASS**

**REC-01**
Steps: Title='Bouillabaisse', instructions, cultural significance, cook_time=45, difficulty=Medium, image upload, click a point on the map (Marseille), tick 2 ingredients + 2 techniques, save.
Expected (P): Redirected to /entry/; recipe visible in list.
**Result: PASS**

**REC-02 (N)**
Steps: Submit with empty title/instructions/cultural_significance.
Expected (P): Red banner: 'Title, instructions and cultural significance are required.'
**Result: PASS**

**REC-03 (N)**
Steps: Submit without clicking the map.
Expected (P): Red banner: 'Pick a location on the map.'
**Result: PASS**

**REC-04**
Steps: Submit with cook_time='abc'.
Expected (P): Recipe created with cook_time=30 (silently coerced — view.py:457-459).
**Result: PASS**

**REC-05**
Steps: Verify recipe's regions automatically set to U1's sharing_region (France) — visit detail page.
Expected (P): 'France' listed under regions.
**Result: PASS**

**4.2 List + filters (/recipes/):**

**REC-10**
Steps: Visit /recipes/; no filters.
Expected (P): Up to 60 recipes listed, newest first.
**Result: PASS**

**REC-11**
Steps: Filter title=bou.
Expected (P): Only 'Bouillabaisse' returned.
**Result: PASS**

**REC-12**
Steps: Filter region=France.

Expected (P): All France-tagged recipes returned.
**Result: PASS**

**REC-13**
Steps: Filter difficulty=Hard.
Expected (P): Only Hard recipes.
**Result: PASS**

**REC-14**
Steps: cook_min=60, cook_max=30 (swapped automatically).
Expected (P): Min/max swapped server-side, filter applied as 30-60.
**Result: PASS**

**REC-15**
Steps: ingredient=tomat (substring).
Expected (P): Recipes referencing tomato-containing ingredients.
**Result: PASS**

**REC-16**
Steps: ingredient=tomatto (typo).
Expected (P): Result set uses fuzzy correction; UI shows 'Showing results for: Tomato' hint.
**Result: PASS**

**REC-17**
Steps: technique=brais (substring).
Expected (P): Recipes using Braising etc.
**Result: PASS**

**REC-18**
Steps: instructions=simmer.
Expected (P): Recipes whose content contains 'simmer'.
**Result: PASS**

**REC-19**
Steps: Combination: region=France&difficulty=Medium&cook_max=60.
Expected (P): Intersection of all filters applied.
**Result: PASS**

**REC-20**
Steps: Open any recipe detail.
Expected (P): Title, image, regions, ingredients, techniques, cultural significance, cook time, difficulty, author, like count, comment section visible.
**Result: PASS**

**REC-21**
Steps: Click Like.
Expected (P): Counter +1, button switches to 'Liked' state.
**Result: PASS**

**REC-22**
Steps: Click Like again.
Expected (P): Counter -1, button reverts.
**Result: PASS**

**TEC-01**
Steps: /techniques/add/ → fill name/description/cultural_significance, optional necessary_tools, save.
Expected (P): Redirected to /techniques/; new entry visible.
**Result: PASS**

**TEC-02 (N)**
Steps: Submit add form missing any required field.
Expected (P): Red banner: 'Name, description and cultural significance are required.'
**Result: PASS**

**TEC-03**
Steps: /techniques/?name=brais (substring).
Expected (P): Matching results.
**Result: PASS**

**TEC-04**
Steps: /techniques/?name=brassing (typo).
Expected (P): Fuzzy correction applied, 'Showing results for: Braising'.
**Result: PASS**

**TEC-05**
Steps: /techniques/?tools=knife.
Expected (P): Filters by necessary_tools.
**Result: PASS**

**TEC-06**
Steps: Visit /techniques/<pk>/.
Expected (P): Detail page renders with regions and comments.
**Result: PASS**

**ING-01**
Steps: /ingredients/add/ → name + description, save.

Expected (P): Redirected to /ingredients/; new entry visible.
**Result: PASS**

**ING-02 (N)**
Steps: Submit add form with empty name or description.
Expected (P): Red banner: 'Name and description are required.'
**Result: PASS**

**ING-03**
Steps: /ingredients/?name=tom (substring).
Expected (P): Tomato-style results.
**Result: PASS**

**ING-04**
Steps: /ingredients/?name=tomatto (typo).
Expected (P): Fuzzy correction shown.
**Result: PASS**

**ING-05**
Steps: /ingredients/?author=erotg.
Expected (P): Author email substring match.
**Result: PASS**

**ING-06**
Steps: Visit /ingredients/<pk>/.
Expected (P): Detail renders, comment section present.
**Result: PASS**

**SCH-01**
Steps: Submit q= (empty).
Expected (P): Three empty result groups, no error.
**Result: PASS**

**SCH-02**
Steps: Submit q=bouilla.
Expected (P): Recipes matching title; ingredients/techniques sections may be empty.
**Result: PASS**

**SCH-03**
Steps: Submit q=cultural.
Expected (P): Hits across all three groups where description / cultural_significance contains the word.
**Result: PASS**

**ROAD-01**
Steps: Visit /culinary-road/.
Expected (P): Interactive map shown; count badge matches number of recipes with lat/long set.
**Result: PASS**

**ROAD-02**
Steps: Click any marker.
Expected (P): Popup shows recipe title, image (if present), cultural significance, link to detail page.
**Result: PASS**

**ROAD-03**
Steps: Add a recipe at a new location (REC-01) then revisit.
Expected (P): New marker appears.
**Result: PASS**

**CMP-01**
Steps: Visit /culinary-compare/ (default kind=recipe).
Expected (P): 'Recipes' tab active; multi-select shows all recipes; empty hint shown ('No entries selected yet…').
**Result: PASS**

**CMP-02**
Steps: Pick 1 recipe, click Compare.
Expected (P): Hint: 'Pick at least two entries to see a side-by-side comparison.'
**Result: PASS**

**CMP-03**
Steps: Pick 2+ recipes, click Compare.
Expected (P): Side-by-side table with Title, Author, Regions, Cook Time, Difficulty, Ingredients, Techniques, Instructions, Cultural Significance.
**Result: PASS**

**CMP-04**
Steps: Switch to Ingredients tab.
Expected (P): Page loads successfully (previously crashed with 500 before the view.py:366 fix). Multi-select shows all ingredients.
**Result: PASS**

**CMP-05**
Steps: Pick 2+ ingredients, Compare.
Expected (P): Table renders with Name, Author, Description, Cultural Significance.
**Result: PASS**

**CMP-06**
Steps: Switch to Techniques tab, pick 2+, Compare.
Expected (P): Table with Name, Author, Regions, Description, Cultural Significance, Necessary Tools.
**Result: PASS**

**CMP-07 (N)**
Steps: Manually craft URL /culinary-compare/?kind=bogus.
Expected (P): Falls back to kind=recipe silently.
**Result: PASS**

**CMP-08 (N)**
Steps: Pass non-numeric ids (?ids=foo&ids=1).
Expected (P): 'foo' ignored, '1' considered — no 500.
**Result: PASS**

**COM-01**
Steps: Submit a top-level comment.
Expected (P): Comment appears immediately, attributed to logged-in user.
**Result: PASS**

**COM-02 (N)**
Steps: Submit an empty comment.
Expected (P): Page reloads, no comment created.
**Result: PASS**

**COM-03**
Steps: Click Reply on a comment, submit a reply body.
Expected (P): Reply nested under the parent; reply count visible.
**Result: PASS**

**COM-04 (N)**
Steps: Try to reply to a reply (not a top-level comment).
Expected (P): Server treats parent as null (one-level threading enforced at view.py:716) — reply becomes a top-level comment.
**Result: PASS**

**COM-05**
Steps: As U2, click Like on a U1 comment.
Expected (P): Like counter +1.
**Result: PASS**

**COM-06**
Steps: Click Like again as U2.

Expected (P): Like counter -1.
**Result: PASS**

**COM-07**
Steps: Click Dislike on a previously liked comment.
Expected (P): Like -1, Dislike +1 (mutual exclusion at view.py:730).
**Result: PASS**

**COM-08**
Steps: Repeat 01-07 on /ingredients/<pk>/ and /techniques/<pk>/.
Expected (P): Same behaviour across all three target kinds.
**Result: PASS**

**EDT-01**
Steps: As U1, open one of U1's recipes, click Edit.
Expected (P): Form pre-filled with current values.
**Result: PASS**

**EDT-02**
Steps: Change title, save.
Expected (P): Detail page reflects new title immediately.
**Result: PASS**

**EDT-03 (N)**
Steps: Clear required field (e.g. title) and save.
Expected (P): Red banner: 'Title is required.'
**Result: PASS**

**EDT-04**
Steps: Edit a technique's necessary_tools to empty and save.
Expected (P): Save succeeds (field is optional — _EDIT_OPTIONAL at view.py:41).
**Result: PASS**

**EDT-05**
Steps: Edit a recipe with cook_time='-3'.
Expected (P): Coerced to 1 (lower bound at view.py:756).
**Result: PASS**

**EDT-10**
Steps: As U2, open one of U1's recipes → Edit; change title; submit.
Expected (P): Redirected to /recipes/<pk>/?proposed=1; recipe title unchanged on disk.
**Result: PASS**

**EDT-11 (N)**
Steps: As U2, click Edit but submit identical values.

Expected (P): Red banner: 'No changes to propose.'
**Result: PASS**

**EDT-12**
Steps: As U1, visit /proposals/.
Expected (P): U2's proposal in Incoming section with a diff (old → new).
**Result: PASS**

**EDT-13**
Steps: U2 visits /proposals/.
Expected (P): Same proposal listed under Outgoing with status=pending.
**Result: PASS**

**EDT-14**
Steps: As U1, click Approve with note.
Expected (P): Status → approved; recipe title now reflects U2's value; decided_at populated.
**Result: PASS**

**EDT-15**
Steps: As U1, repeat EDT-10 from U2, click Reject with note.
Expected (P): Status → rejected; recipe unchanged.
**Result: PASS**

**EDT-16 (N)**
Steps: As U1, manually POST /proposals/<id>/decide/ against someone else's proposal.
Expected (P): HTTP 403 'Only the entry owner can decide.'
**Result: PASS**

**EDT-17 (N)**
Steps: Approve an already-decided proposal.
Expected (P): Silently redirects to /proposals/ (idempotency at view.py:864-865).
**Result: PASS**

**RPT-01**
Steps: As U2, open U1's recipe → Report → reason 'Spam' → submit.
Expected (P): Redirected to /recipes/<pk>/?reported=1.
**Result: PASS**

**RPT-02 (N)**
Steps: Submit report with empty reason.
Expected (P): Red banner: 'Please describe why you are reporting this entry.'
**Result: PASS**

**RPT-03**
Steps: Repeat for an ingredient and a technique.

Expected (P): Each Report row created with the correct polymorphic FK.
**Result: PASS**

**RPT-10 (N)**
Steps: As U1 (non-staff), visit /reports/.
Expected (P): HTTP 403 'Admin access only.'
**Result: PASS**

**RPT-11**
Steps: As ADMIN, visit /reports/.
Expected (P): Pending reports + recent (resolved/dismissed) sections visible.
**Result: PASS**

**RPT-20**
Steps: As ADMIN, on a pending report click Dismiss with a note.
Expected (P): Status → dismissed; resolution_note saved; vanishes from pending.
**Result: PASS**

**RPT-21**
Steps: As ADMIN, Ban User action.
Expected (P): Target entry's author is_active=False; the user can no longer log in (verify AUTH-04 fails for them).
**Result: PASS**

**RPT-22**
Steps: As ADMIN, Delete Entry action.
Expected (P): Target row deleted; a new resolved Report row created for the audit trail (view.py:942-946); detail page returns 404.
**Result: PASS**

**RPT-23 (N)**
Steps: As ADMIN, action an already-resolved report.
Expected (P): Silently redirects to /reports/.
**Result: PASS**